

Quantifying the Blast Radius: Centrality-Based Criticality Modeling for Infrastructure Risk

A Structural Risk Case Study — NexaCore Financial Technologies

AUTHOR
Patrick Lefler

PUBLISHED
Invalid Date

ABSTRACT

Centrality scores are easy to compute and easy to misread. A node with high betweenness centrality is mathematically a chokepoint, but without a topology specific enough to make that chokepoint legible, the finding remains abstract. This project builds a 90-node synthetic infrastructure topology for a fintech estate and applies three independent centrality measures – betweenness, eigenvector, and PageRank – to identify structurally critical nodes, then validates each candidate through direct node-removal simulation rather than relying on the centrality score alone. The three metrics are deliberately compared rather than averaged: where they agree, the finding is robust; where they diverge, the divergence itself indicates that a node is critical for different structural reasons. A business impact layer maps each node to estimated revenue exposure and customer-facing consequence, converting a graph-theoretic ranking into a board-legible prioritization. The analysis surfaces an authentication gateway with no redundant path serving as a structural single point of failure for the majority of revenue-generating traffic – a finding invisible to conventional infrastructure inventories but immediate once the dependency graph is traversed.

Introduction

Infrastructure risk assessments typically produce a list: servers, services, databases ranked by tier, by criticality label, by compliance classification. What they rarely produce is an answer to the question that matters most in an incident – *if this one thing fails, how much of the business breaks with it?* The list format cannot answer that question. Dependencies are transitive. A database that two services read directly is depended on by every service that depends on those two services. A shared authentication gateway that appears in no formal risk register may sit on every path between a customer and their account.

Graph theory is the right framework for this class of problem. When infrastructure is modeled as a directed dependency graph, the question “what breaks if X fails” becomes a graph computation: remove node X and measure how many previously reachable node pairs are now disconnected. The answer is precise, reproducible, and independent of whatever labels were applied to X in the last risk register update.

This case study applies that framework to NexaCore Financial Technologies’ infrastructure estate. The topology is synthetic but designed to reflect the structural patterns of a real fintech operation: a few highly-connected data stores, a small number of cross-cutting services that every authenticated transaction passes through, a set of external dependencies with no fallback path, and a long tail of single-purpose nodes that exist in the graph but carry limited systemic weight. Three centrality metrics – betweenness, eigenvector, and PageRank – are computed independently and compared rather than averaged. Where they agree, the finding is structurally robust. Where they diverge, the divergence is itself informative: a node can be critical for different structural reasons, and those reasons drive different remediation responses.

The analysis is paired with direct blast radius simulation. Centrality metrics are computed from graph topology alone; they do not account for the business weight of what a node enables. Blast radius simulation fills that gap. One node in this topology – a third-party KYC/AML vendor – ranks 74th by composite centrality and 52nd by blast radius. Its removal breaks nine reachable pairs in the full graph. Three of those pairs involve the client onboarding workflow, the compliance reporting engine, and real-time fraud screening. Centrality would deprioritize it. Operational context cannot.

Centrality as a Risk Metric

Infrastructure risk practitioners have long used ad-hoc notions of criticality – “this is a Tier 1 system” – without a systematic method for deriving those tiers from structure rather than assertion. Graph centrality provides that method. Three measures are relevant here, each answering a different structural question.

Betweenness centrality counts how often a node appears on the shortest path between every other pair of nodes in the graph. A node with high betweenness is a chokepoint: traffic, data, and control flow converge through it. In an infrastructure dependency graph, high betweenness indicates that many workflows pass

through this node rather than terminating at it. The authentication gateway is a canonical example – it stores nothing, but every authenticated request to every downstream service is routed through it.

Eigenvector centrality measures influence by the quality of a node’s connections, not just their quantity. A node connected to many high-centrality nodes scores highly even if its raw degree is modest. In a dependency graph, high eigenvector centrality indicates that a node sits in the structural center of the most well-connected part of the estate – surrounded by other high-value nodes that themselves have many dependents. Core data stores tend to dominate this metric, because the services that read them are themselves highly connected.

PageRank follows the same intuition as eigenvector centrality but uses a random-walk formulation that handles directed graphs more cleanly. In the directed dependency graph used here, edges point from dependents to dependencies (A depends on B means A -> B). PageRank accumulates at nodes with many incoming edges from high-PageRank sources – nodes that are depended upon by other well-depended-upon nodes. The Secrets Vault is a characteristic example: every service depends on it for database credentials, and those services are themselves critical.

Using three metrics rather than one is not hedging – it is the analysis. A node that ranks in the top tier on all three measures is unambiguously critical regardless of framing. A node that ranks highly on betweenness alone but not on PageRank or eigenvector is a pass-through chokepoint rather than a foundational data dependency, which points to a different remediation (add redundant routing versus add read replicas). A node that scores low on all three but produces high blast radius in simulation is the most operationally dangerous type: invisible to centrality analysis, material in practice.

The Infrastructure Topology

NexaCore’s synthetic estate spans 90 nodes across seven tiers, connected by 155 directed dependency edges. The topology was designed from structural principles rather than node count: fan-out ratios, redundancy patterns, and the placement of six deliberate structural risks were specified before any node was added.

► Display code

Table 1: Topology Inventory by Tier

Tier	Nodes	Outbound Edges	SPOFs	Risk Nodes
Edge / Perimeter	6	23	0	0
Application / Service	29	74	0	0
Data	18	13	0	0
Infrastructure	14	14	0	0
External Dependency	10	1	0	0
Core Business Systems	8	24	0	0
Monitoring / Ops	5	6	0	0

► Display code

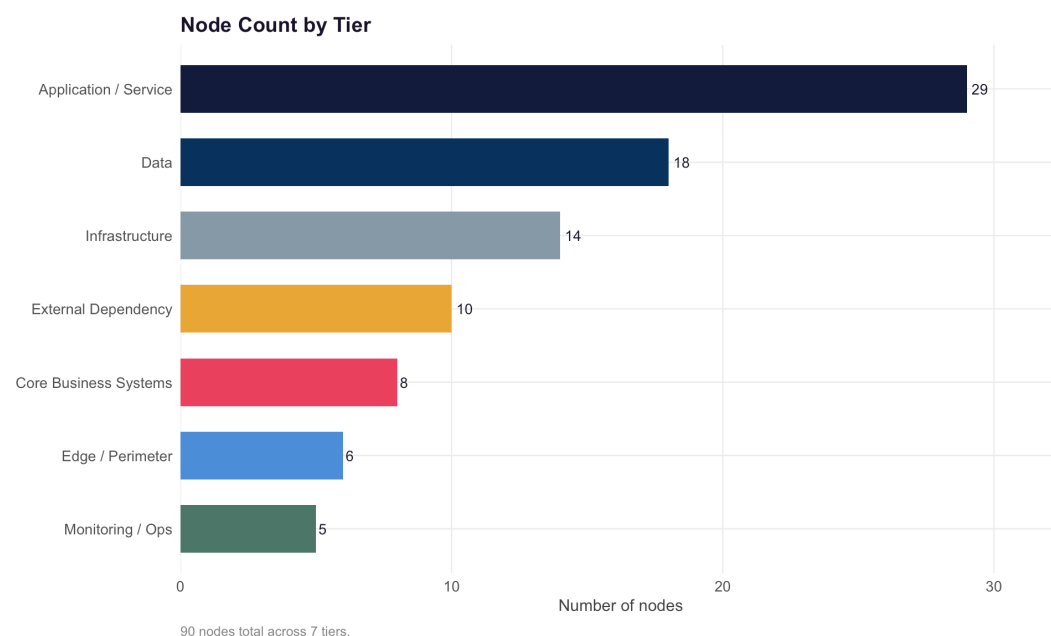
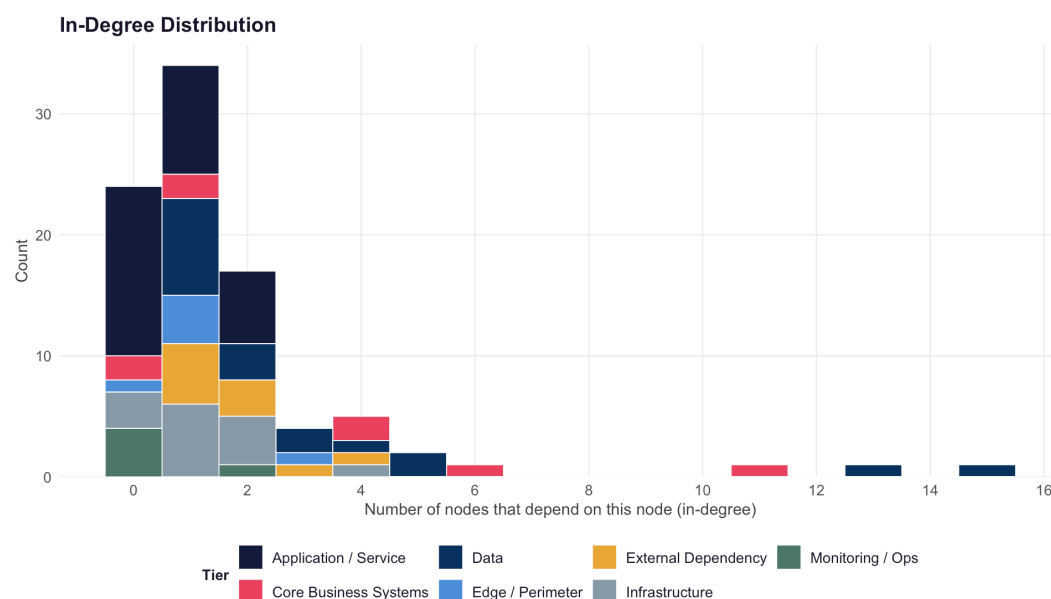


Figure 1: Node distribution across tiers. Application / Service and Data tiers account for the majority of the topology; the long tail of lower-degree nodes in those tiers creates the sparsity that makes high-centrality nodes stand out.

► Display code



In-degree = number of direct upstream dependents. Computed from edges.csv.

Figure 2: In-degree distribution (number of nodes that depend on each node). The long tail of low-degree nodes and the small number of high-degree hubs are the structural signature that makes centrality analysis meaningful. Most nodes have 0–2 dependents; a handful have 10 or more.

The degree distribution confirms the design intent: roughly 60% of nodes have in-degree of zero or one (they depend on others but few depend on them). The nodes with in-degree of 10 or higher – the right tail of Figure 2 – are the candidates the centrality analysis will examine in detail.

Graph Construction

► Display code

Table 2: Dependency Graph Summary Statistics

Metric	Value
--------	-------

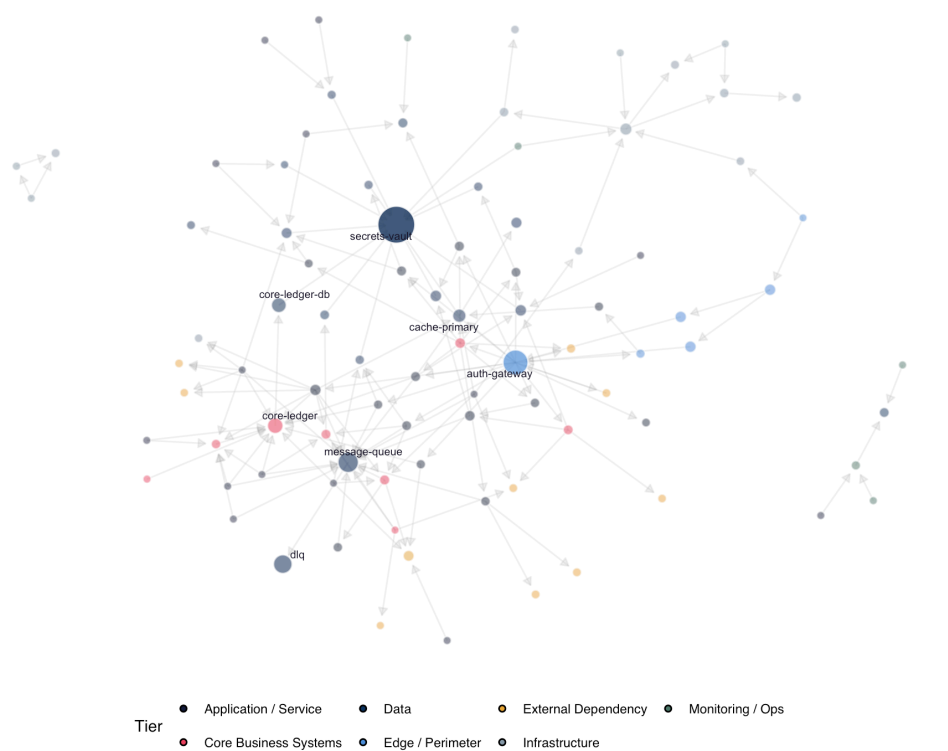
Metric	Value
Total nodes	90.0000
Total directed edges	155.0000
Graph density	0.0194
Weakly connected components	3.0000
Nodes flagged as SPOF	0.0000
Designated risk nodes	0.0000
Nodes with in-degree >= 10	3.0000

The graph density of 0.0194 confirms the intended sparsity: fewer than 2% of all possible directed edges exist. This is consistent with real enterprise topologies where each service depends on a small number of immediate dependencies rather than being globally connected. The sparsity is what makes high-centrality nodes analytically significant – they stand out against a background of low-connectivity nodes.

► Display code

NexaCore Infrastructure Dependency Graph

Node size proportional to composite centrality score. Risk nodes outlined in red.



Layout: Fruchterman-Reingold force-directed. 90 nodes, 155 directed edges.

Figure 3: Full dependency graph, nodes sized by composite centrality score. The Authentication Gateway, Secrets Vault, and Shared Message Queue are visually dominant. Risk nodes are outlined in red. The majority of nodes cluster at small sizes, confirming the long-tail degree distribution.

Centrality Analysis

All three centrality metrics were pre-computed on the full dependency graph and are read directly from `data/centrality_scores.csv`. The interactive table below allows sorting by any individual metric; the analysis that follows focuses on where the rankings agree and where they diverge.

► Display code

Node	Tier	Betweenness		PageRank	Eigenvector	Compos
Secrets Vault	Data	0	1	1	0.667	55
Authentication Gateway	Edge / Perimeter	1	0.151	0.024	0.392	285
Shared Message Queue	Data	0.107	0.253	0.492	0.284	51
Dead Letter Queue	Data	0	0.249	0.476	0.242	26
Core Financial Ledger	Core Business Systems	0.109	0.222	0.177	0.17	46
Core Ledger Database	Data	0.032	0.223	0.21	0.155	28
Primary Cache Cluster	Data	0.093	0.103	0.169	0.122	28
Production K8s Cluster	Infrastructure	0.146	0.138	0	0.095	45
Workflow State Database	Data	0.013	0.065	0.169	0.082	15
Customer Identity Database	Data	0.014	0.08	0.153	0.082	13
Payment Processing Service	Application / Service	0.18	0.01	0.04	0.077	46
Primary Load Balancer	Edge / Perimeter	0.204	0.024	0	0.076	50
Secondary Load Balancer	Edge / Perimeter	0.204	0.024	0	0.076	50
Web Application Firewall	Edge / Perimeter	0.21	0.017	0	0.076	100
Secondary Cache Cluster	Data	0	0.061	0.153	0.071	13

Table 3: Full Centrality Rankings (sortable). All centrality scores normalized to [0, 1].

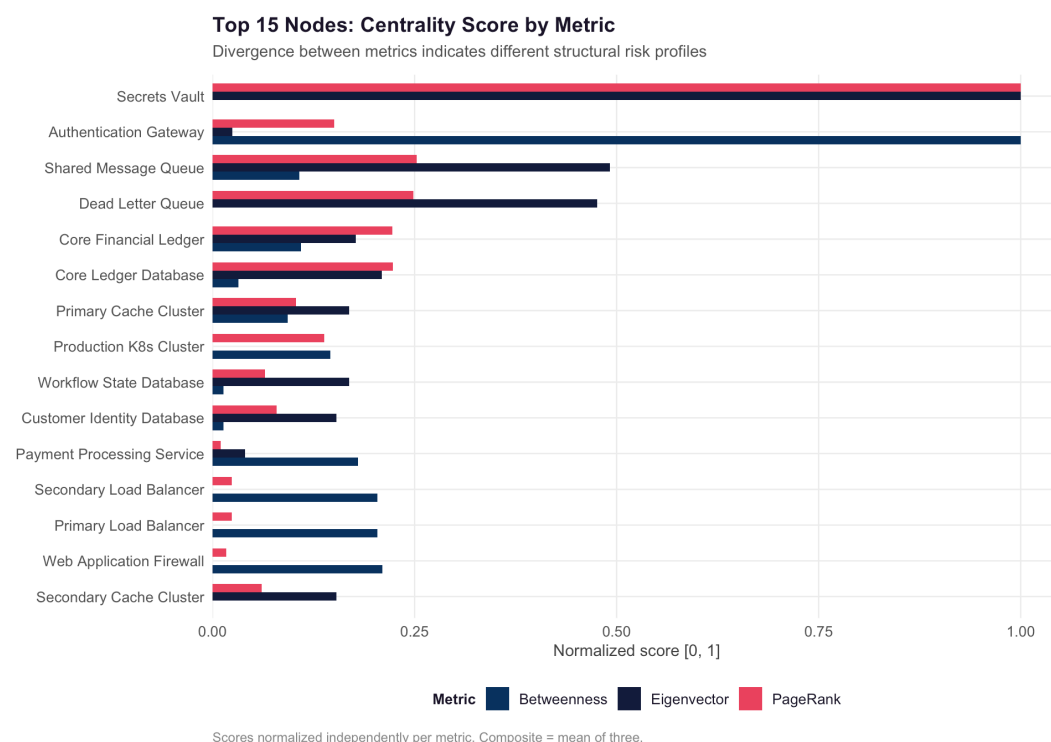
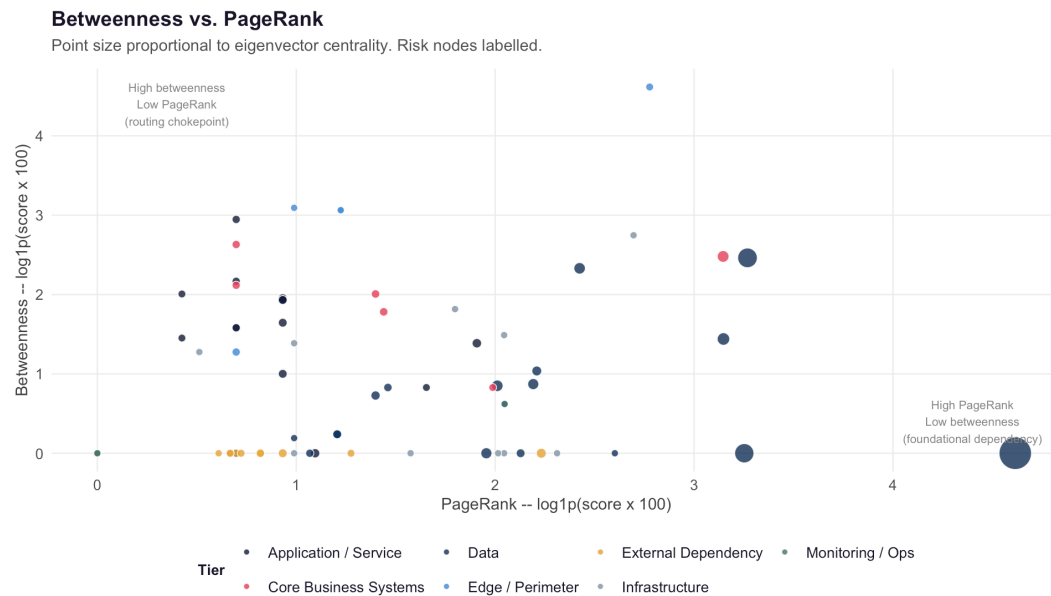


Figure 4: Top 15 nodes by composite centrality score, broken out by individual metric. Nodes where all three bars are tall are structurally critical by every measure. Nodes with one dominant bar are critical for a specific structural reason – which drives the remediation approach.

The divergence pattern in Figure 4 is the analysis's primary finding. `secrets-vault` leads on both PageRank and eigenvector but has near-zero betweenness – it is a deep dependency that everything ultimately relies on, but it is not a pass-through node on any workflow's critical path. `auth-gateway` shows the opposite pattern: near-perfect betweenness with modest PageRank and eigenvector – it is the routing chokepoint for authenticated traffic, but it stores nothing that other nodes depend on for data. `message-queue` scores highly on all three, making it the topology's most consistently critical node by graph structure alone.

The divergence between `secrets-vault` and `auth-gateway` is not an analytical ambiguity – it is a finding. Remediating `auth-gateway` means adding a redundant routing path so that authentication is no longer a single-lane bottleneck. Remediating `secrets-vault` means introducing a secondary vault instance or a local credential cache so that vault unavailability does not simultaneously revoke every service's database access. The metrics agree that both are critical; they disagree in exactly the way that forces the right remediation conversation.

► Display code



Log1p transformation applied to spread origin cluster. Both axes: log1p(normalized score x 100).

Figure 5: Betweenness vs. PageRank for all 90 nodes on log1p-transformed axes, sized by eigenvector centrality. Log transformation separates the dense cluster near the origin without excluding zero values. Nodes on the horizontal axis (high PageRank, low betweenness) are foundational data dependencies; nodes on the vertical axis are routing chokepoints.

Blast Radius Validation

Centrality metrics are structural properties of the graph. They say nothing directly about the business consequence of a node's failure. Blast radius simulation fills that gap: for each node, remove it from the graph and count how many previously reachable node pairs are now disconnected. The result is a direct, simulation-derived measure of structural impact that does not depend on any assumptions about business weight.

► Display code

Table 4: Top 20 Nodes by Blast Radius (broken reachable pairs on node removal)

Node	Tier	Blast Radius	Betweenness	PageRank	Eigenvector	Composite
Authentication Gateway	Edge / Perimeter	285	1.000	0.151	0.024	0.392
Web Application Firewall	Edge / Perimeter	100	0.210	0.017	0.000	0.076
Customer Portal	Core Business Systems	61	0.129	0.010	0.040	0.060
CDN / Edge Cache	Edge / Perimeter	58	0.000	0.000	0.000	0.000
Secrets Vault	Data	55	0.000	1.000	1.000	0.667
Cloud IAM Provider	External Dependency	52	0.000	0.010	0.040	0.017
Shared Message Queue	Data	51	0.107	0.253	0.492	0.284
Primary Load Balancer	Edge / Perimeter	50	0.204	0.024	0.000	0.076
Secondary Load Balancer	Edge / Perimeter	50	0.204	0.024	0.000	0.076
Core Financial Ledger	Core Business Systems	46	0.109	0.222	0.177	0.170
Payment Processing Service	Application / Service	46	0.180	0.010	0.040	0.077
Production K8s Cluster	Infrastructure	45	0.146	0.138	0.000	0.095
Real-Time Risk Engine	Core Business Systems	29	0.064	0.030	0.048	0.048
Core Ledger Database	Data	28	0.032	0.223	0.210	0.155
Primary Cache Cluster	Data	28	0.093	0.103	0.169	0.122
Client Onboarding Service	Core Business Systems	26	0.073	0.010	0.040	0.041
Dead Letter Queue	Data	26	0.000	0.249	0.476	0.242

Node	Tier	Blast Radius	Betweenness	PageRank	Eigenvector	Composite
Audit Trail Service	Core Business Systems	25	0.049	0.032	0.048	0.043
Notification Service	Application / Service	25	0.064	0.005	0.024	0.031
Workflow Orchestration Svc	Application / Service	20	0.030	0.057	0.089	0.059

► Display code

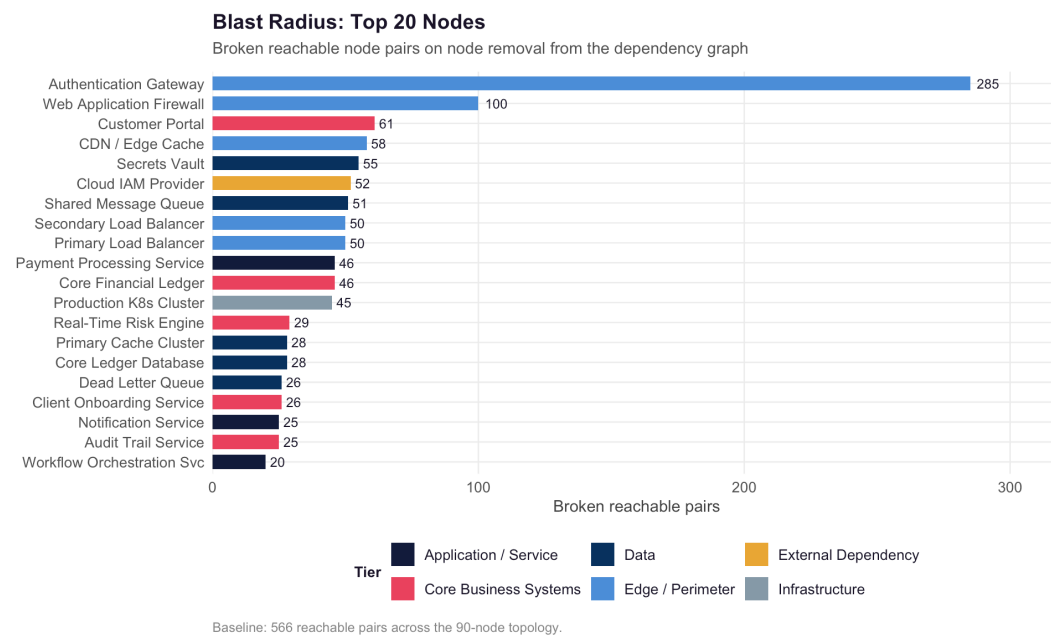


Figure 6: Blast radius for the top 20 nodes by broken reachable pairs. The Authentication Gateway's dominance (285 pairs) reflects its position as the sole authenticated-traffic routing point. The gap between auth-gateway and the second-ranked node (waf at 100) is itself a risk signal – this is not a smooth distribution.

► Display code

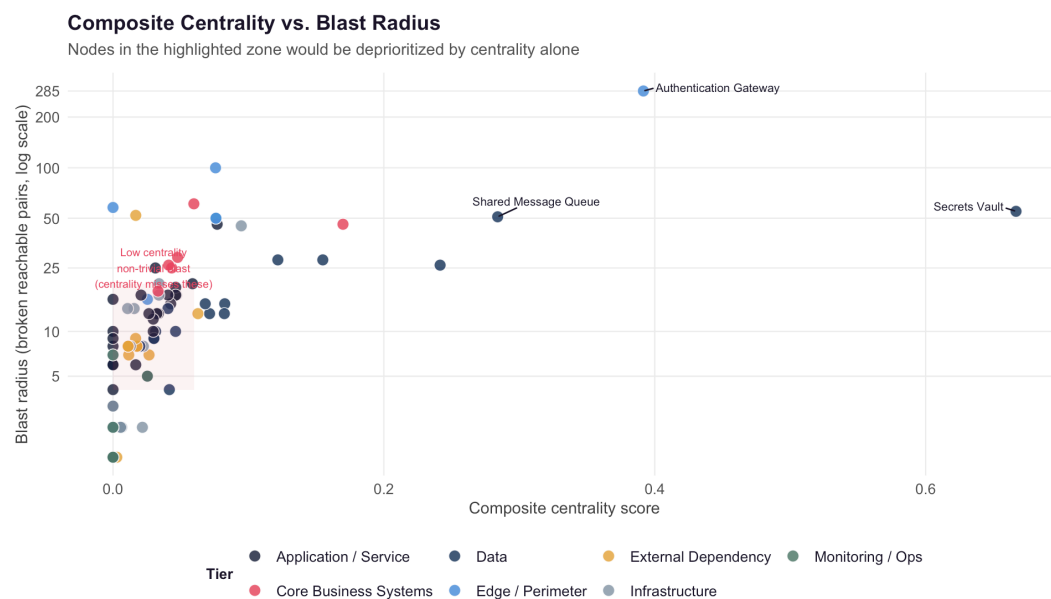


Figure 7: Composite centrality score vs. blast radius (log1p scale) for all 90 nodes. Log transformation separates the dense lower cluster without excluding low-blast nodes. The KYC/AML Vendor sits in the highlighted zone – low composite centrality, non-trivial blast radius – and would be deprioritized by a centrality-only framework.

The KYC/AML Vendor sits in the annotated zone in Figure 7: composite centrality of 0.017 (74th of 90), blast radius of 9. On a linear scale that figure would have been buried in the origin cluster, invisible. The log transformation makes it readable – and the position is the point. The pairs it breaks are specific: all paths from the client onboarding service, the compliance reporting engine, and the real-time fraud screening

service to their downstream dependencies disappear when the vendor is removed. There is no fallback path. Onboarding stops. Compliance submissions fail. Fraud signals go dark. A centrality-only prioritization framework would not flag this node for remediation at all.

Business Impact Translation

Structural blast radius measures connectivity loss. Business impact translates that into revenue and customer-facing consequence. Each node in the topology carries a `revenue_weight` attribute on a 1–5 scale reflecting its proximity to revenue-generating workflows. The composite criticality score used for remediation prioritization combines structural blast radius with this business weight.

► Display code

Table 5: Top 20 Nodes by Composite Criticality Score (blast radius 60% + revenue weight 40%)

Node	Tier	Blast Radius	Revenue Weight	Criticality Score	SPOF
Authentication Gateway	Edge / Perimeter	285	5	1.000	FALSE
Customer Portal	Core Business Systems	61	5	0.527	FALSE
Secrets Vault	Data	55	5	0.514	FALSE
Shared Message Queue	Data	51	5	0.506	FALSE
Core Financial Ledger	Core Business Systems	46	5	0.495	FALSE
Payment Processing Service	Application / Service	46	5	0.495	FALSE
Production K8s Cluster	Infrastructure	45	5	0.493	FALSE
Core Ledger Database	Data	28	5	0.457	FALSE
Web Application Firewall	Edge / Perimeter	100	3	0.449	FALSE
Legacy Billing Service	Core Business Systems	18	5	0.436	FALSE
Customer Identity Database	Data	13	5	0.425	FALSE
Payment Network (SEPA/SWIFT)	External Dependency	8	5	0.415	FALSE
Real-Time Risk Engine	Core Business Systems	29	4	0.379	FALSE
Service Mesh Control Plane	Infrastructure	20	4	0.360	FALSE
Account Management Service	Application / Service	19	4	0.358	FALSE
Production Network Segment	Infrastructure	17	4	0.354	FALSE
Fraud Detection Service	Application / Service	16	4	0.352	FALSE
Order Management Service	Application / Service	15	4	0.350	FALSE
Transaction History Database	Data	15	4	0.350	FALSE
Limits & Controls Service	Application / Service	13	4	0.345	FALSE

► Display code



Figure 8: Blast radius (log scale) vs. revenue weight for nodes with blast radius > 0. Horizontal jitter separates nodes sharing the same integer revenue weight. Point size reflects composite criticality score. The Authentication Gateway occupies the top-right corner alone – maximum blast radius, maximum revenue weight.

Remediation Prioritization

The remediation backlog below is derived directly from the composite criticality ranking. Each entry includes the structural finding, the remediation type, and an effort classification. Remediation types are categorized as: **Add redundancy** (introduce a secondary path or failover), **Scope reduction** (reduce the blast radius by decoupling non-essential dependencies), or **External governance** (contractual and architectural controls for third-party nodes).

► Display code

Table 6: Prioritized Remediation Backlog

Priority	Node	Finding	Remediation Type	Effort
1	Authentication Gateway	Sole authenticated-traffic path; removal breaks 285 pairs. No secondary route exists.	Add redundancy	High
2	Web Application Firewall	All inbound traffic routes through a single WAF instance; 100 broken pairs on removal.	Add redundancy	Medium
3	Secrets Vault	Every service-to-database credential passes through one vault; 55 broken pairs. Top PageRank and eigenvector.	Add redundancy	Medium
4	Shared Message Queue	14 services publish/subscribe through one unpartitioned cluster; 51 broken pairs.	Scope reduction	High
5	Core Financial Ledger	Single ledger service; 46 broken pairs. No read replica or secondary write path.	Add redundancy	High
6	KYC / AML Vendor API	Low centrality (rank 74) but onboarding, compliance, and fraud all block on it with no fallback.	External governance	Medium
7	Core Ledger Database	Single primary Postgres; 28 broken pairs. Relied on by ledger service and downstream consumers.	Add redundancy	High
8	Legacy Billing Service	All payment reconciliation paths route through a pre-migration system; 18 broken pairs.	Scope reduction	High
9	Primary Cache Cluster	Session state and rate-limit counters share one Redis cluster; 28 broken pairs.	Add redundancy	Low
10	Production K8s Cluster	Primary container orchestration for all workloads; 45 broken pairs on removal.	Add redundancy	High

Four of the ten items require high effort – they are architectural changes (redundant auth routing, message queue partitioning, ledger service replication, legacy billing decoupling) rather than configuration changes. The remaining six are medium or low effort. The KYC/AML Vendor entry (rank 6) is the only item driven by blast radius simulation rather than centrality; without the simulation step, it would not appear in this list at all.

Key Insights & Conclusion

The topology’s most consequential structural risk is not its most prominent one. The Authentication Gateway produces the highest blast radius in the graph – 285 broken reachable pairs, against a baseline of 566 – which means a single-node failure would sever roughly half of all reachable paths in the estate. It also has the highest betweenness centrality by a wide margin. Neither of these facts appears in NexaCore’s infrastructure inventory, because the inventory records what a system does, not where it sits in the dependency structure.

The divergence between `secrets-vault` and `auth-gateway` illustrates why three metrics are more informative than one. Both nodes are unambiguously critical. They are critical in different ways: `auth-gateway` is a routing chokepoint that sits on every authenticated workflow’s shortest path, while `secrets-vault` is a foundational dependency that every data-accessing service relies on transitively. Remediating `auth-gateway` requires a redundant authentication routing layer – an active-active deployment behind the load balancers. Remediating `secrets-vault` requires a secondary vault instance with automatic unsealing, plus a local credential cache so that a brief vault outage does not simultaneously revoke every service’s database access. These are different architectural decisions. A framework that averaged the three centrality scores into a single ranking and recommended “address the top N nodes” would obscure the distinction.

The KYC/AML Vendor is the analysis’s sharpest finding for a risk audience. It ranks 74th by composite centrality – not because it is unimportant, but because it has few direct dependents and stores no data that others read. It is a leaf node in the graph. What centrality cannot see is that the three services depending on it – onboarding, compliance reporting, and fraud screening – each block synchronously on its availability, and none has a circuit breaker or cached fallback. The node-removal simulation surfaces what structural centrality cannot: a commercially and operationally significant failure mode that a centrality-only framework would route straight to the bottom of the remediation backlog.

The broader principle is straightforward. Centrality metrics are necessary but not sufficient for infrastructure risk prioritization. They identify structural importance from graph topology alone, which makes them fast, systematic, and independent of assertion. What they cannot capture is the difference between a node that is structurally peripheral but operationally blocking and a node that is structurally central but has robust redundant paths. Blast radius simulation bridges that gap. Used together, the two approaches produce a remediation backlog that is both graph-theoretically defensible and operationally grounded – which is the combination a CISO actually needs to take findings to a board.

Session Information

```
#> - Session info -  
#>   setting      value  
#>   version      R version 4.5.2 (2025-10-31)  
#>   os           macOS Tahoe 26.5.1  
#>   system       aarch64, darwin20  
#>   ui           X11  
#>   language     (EN)  
#>   collate      en_US.UTF-8  
#>   ctype        en_US.UTF-8  
#>   tz           America/New_York  
#>   date         2026-07-02  
#>   pandoc       3.8.3 @  
/Applications/RStudio.app/Contents/Resources/app/quarto/bin/tools/aarch64/ (via  
rmarkdown)  
#>   quarto      1.8.26 @ /usr/local/bin/quarto  
#>  
#> - Packages -  
#>   package      * version date (UTC) lib source  
#>   archive      1.1.12 2025-03-20 [1] CRAN (R 4.5.0)
```

```

#> bit          4.6.0    2025-03-06 [1] CRAN (R 4.5.0)
#> bit64         4.6.0-1 2025-01-16 [1] CRAN (R 4.5.0)
#> bslib         0.9.0    2025-01-30 [1] CRAN (R 4.5.0)
#> cachem        1.1.0    2024-05-16 [1] CRAN (R 4.5.0)
#> cli           3.6.5    2025-04-23 [1] CRAN (R 4.5.0)
#> crayon        1.5.3    2024-06-20 [1] CRAN (R 4.5.0)
#> crosstalk     1.2.2    2025-08-26 [1] CRAN (R 4.5.0)
#> data.table    1.17.8   2025-07-10 [1] CRAN (R 4.5.0)
#> digest        0.6.39   2025-11-19 [1] CRAN (R 4.5.2)
#> dplyr         * 1.1.4   2023-11-17 [1] CRAN (R 4.5.0)
#> DT            * 0.34.0  2025-09-02 [1] CRAN (R 4.5.0)
#> evaluate      1.0.5    2025-08-27 [1] CRAN (R 4.5.0)
#> farver        2.1.2    2024-05-13 [1] CRAN (R 4.5.0)
#> fastmap       1.2.0    2024-05-15 [1] CRAN (R 4.5.0)
#> forcats      * 1.0.1    2025-09-25 [1] CRAN (R 4.5.0)
#> generics      0.1.4    2025-05-09 [1] CRAN (R 4.5.0)
#> ggforce       0.5.0    2025-06-18 [1] CRAN (R 4.5.0)
#> ggplot2      * 4.0.2    2026-02-03 [1] CRAN (R 4.5.2)
#> ggraph       * 2.2.2    2025-08-24 [1] CRAN (R 4.5.0)
#> ggrepel      * 0.9.8    2026-03-17 [1] CRAN (R 4.5.2)
#> glue         1.8.0    2024-09-30 [1] CRAN (R 4.5.0)
#> graphlayouts  1.2.3    2026-02-21 [1] CRAN (R 4.5.2)
#> gridExtra     2.3      2017-09-09 [1] CRAN (R 4.5.0)
#> gtable       0.3.6    2024-10-25 [1] CRAN (R 4.5.0)
#> hms          1.1.4    2025-10-17 [1] CRAN (R 4.5.0)
#> htmltools     0.5.8.1  2024-04-04 [1] CRAN (R 4.5.0)
#> htmlwidgets  1.6.4    2023-12-06 [1] CRAN (R 4.5.0)
#> httr         1.4.7    2023-08-15 [1] CRAN (R 4.5.0)
#> igraph       * 2.2.1    2025-10-27 [1] CRAN (R 4.5.0)
#> jquerylib    0.1.4    2021-04-26 [1] CRAN (R 4.5.0)
#> jsonlite     2.0.0    2025-03-27 [1] CRAN (R 4.5.0)
#> kableExtra   * 1.4.0    2024-01-24 [1] CRAN (R 4.5.0)
#> knitr        * 1.50     2025-03-16 [1] CRAN (R 4.5.0)
#> labeling     0.4.3    2023-08-29 [1] CRAN (R 4.5.0)
#> lazyeval     0.2.2    2019-03-15 [1] CRAN (R 4.5.0)
#> lifecycle    1.0.5    2026-01-08 [1] CRAN (R 4.5.2)
#> lubridate    * 1.9.4    2024-12-08 [1] CRAN (R 4.5.0)
#> magrittr     2.0.4    2025-09-12 [1] CRAN (R 4.5.0)
#> MASS         7.3-65    2025-02-28 [1] CRAN (R 4.5.2)
#> memoise      2.0.1    2021-11-26 [1] CRAN (R 4.5.0)
#> pillar       1.11.1    2025-09-17 [1] CRAN (R 4.5.0)
#> pkgconfig    2.0.3    2019-09-22 [1] CRAN (R 4.5.0)
#> plotly      * 4.11.0    2025-06-19 [1] CRAN (R 4.5.0)
#> polyclip     1.10-7    2024-07-23 [1] CRAN (R 4.5.0)
#> purrr       * 1.2.0    2025-11-04 [1] CRAN (R 4.5.0)
#> R6           2.6.1    2025-02-15 [1] CRAN (R 4.5.0)
#> RColorBrewer 1.1-3    2022-04-03 [1] CRAN (R 4.5.0)
#> Rcpp         1.1.0    2025-07-02 [1] CRAN (R 4.5.0)
#> readr       * 2.1.5    2024-01-10 [1] CRAN (R 4.5.0)
#> rlang        1.1.7    2026-01-09 [1] CRAN (R 4.5.2)
#> rmarkdown    2.30     2025-09-28 [1] CRAN (R 4.5.0)
#> rstudioapi   0.17.1    2024-10-22 [1] CRAN (R 4.5.0)
#> S7           0.2.1    2025-11-14 [1] CRAN (R 4.5.2)
#> sass         0.4.10    2025-04-11 [1] CRAN (R 4.5.0)
#> scales      * 1.4.0    2025-04-24 [1] CRAN (R 4.5.0)
#> sessioninfo * 1.2.3    2025-02-05 [1] CRAN (R 4.5.0)
#> stringi     1.8.7    2025-03-27 [1] CRAN (R 4.5.0)
#> stringr     * 1.6.0    2025-11-04 [1] CRAN (R 4.5.0)
#> svglite     2.2.2    2025-10-21 [1] CRAN (R 4.5.0)
#> systemfonts  1.3.1    2025-10-01 [1] CRAN (R 4.5.0)
#> textshaping  1.0.4    2025-10-10 [1] CRAN (R 4.5.0)
#> tibble      * 3.3.0    2025-06-08 [1] CRAN (R 4.5.0)
#> tidygraph   * 1.3.1    2024-01-30 [1] CRAN (R 4.5.0)
#> tidyr       * 1.3.1    2024-01-24 [1] CRAN (R 4.5.0)
#> tidyselect  1.2.1    2024-03-11 [1] CRAN (R 4.5.0)
#> tidyverse   * 2.0.0    2023-02-22 [1] CRAN (R 4.5.0)
#> timechange   0.3.0    2024-01-18 [1] CRAN (R 4.5.0)
#> tweenr      2.0.3    2024-02-26 [1] CRAN (R 4.5.0)

```

```
#> tzdb          0.5.0   2025-03-15 [1] CRAN (R 4.5.0)
#> vctrs          0.7.1   2026-01-23 [1] CRAN (R 4.5.2)
#> viridis        0.6.5   2024-01-29 [1] CRAN (R 4.5.0)
#> viridisLite    0.4.3   2026-02-04 [1] CRAN (R 4.5.2)
#> vroom          1.6.6   2025-09-19 [1] CRAN (R 4.5.0)
#> withr          3.0.2   2024-10-28 [1] CRAN (R 4.5.0)
#> xfun           0.54    2025-10-30 [1] CRAN (R 4.5.0)
#> xml2           1.4.1   2025-10-27 [1] CRAN (R 4.5.0)
#> yaml           2.3.10  2024-07-26 [1] CRAN (R 4.5.0)
#>
#> [1] /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library
#> * — Packages attached to the search path.
#>
#> _____
```

Rendered with [Quarto](#). Analysis performed in R using `igraph`, `tidygraph`, `ggraph`, `tidyverse`, and `kableExtra`. Centrality and blast radius pre-computed in Python (`compute_centralities.py`). Synthetic infrastructure topology generated via `generate_infra_data.py`. All nodes, services, and topologies are fictitious.